

HERANÇA, REESCRITA E POLIMORFISMO

"O homem absurdo é aquele que nunca muda." -- Georges Clemenceau

Ao final deste capítulo, você será capaz de:

- Dizer o que é herança e quando utilizá-la;
- Reutilizar código escrito anteriormente;
- Criar classes filhas e reescrever métodos;
- Usar todo o poder que o polimorfismo oferece.

9.1 REPETINDO CÓDIGO?

Como toda empresa, nosso banco tem funcionários. Modelemos a classe `Funcionario` :

```
public class Funcionario {  
    private String nome;  
    private String cpf;  
    private double salario;  
    // métodos devem vir aqui  
}
```

Além de um funcionário comum, há também outros cargos, como os gerentes; estes guardam a mesma informação que um funcionário comum, mas também têm outros dados e funcionalidades um pouco diferentes. Por exemplo, um gerente no nosso banco tem uma senha numérica que permite o acesso ao sistema interno do banco, além do número de funcionários os quais ele gerencia:

```
public class Gerente {  
    private String nome;  
    private String cpf;  
    private double salario;  
    private int senha;  
    private int numeroDeFuncionariosGerenciados;  
  
    public boolean autentica(int senha) {  
        if (this.senha == senha) {  
            System.out.println("Acesso Permitido!");  
            return true;  
        } else {  
            System.out.println("Acesso Negado!");  
            return false;  
        }  
    }  
}
```

```

}

// outros métodos
}

```

PRECISAMOS MESMO DE OUTRA CLASSE?

Poderíamos ter deixado a classe `Funcionario` mais genérica, mantendo nela a senha de acesso e o número de funcionários gerenciados. Caso o funcionário não fosse um gerente, deixaríamos esses atributos vazios.

Essa é uma possibilidade, porém, com o tempo, podemos passar a ter muito atributos opcionais, e a classe ficaria estranha. E em relação aos métodos? A classe `Gerente` tem o método `autentica`, que não faz sentido existir em um funcionário o qual não é gerente.

Se tivéssemos um outro tipo de funcionário que tem características diferentes do funcionário comum, precisaríamos criar uma outra classe e copiar o código novamente!

Além disso, se um dia precisarmos adicionar uma nova informação a todos os funcionários, precisaremos passar por todas as classes de funcionário e adicionar esse atributo. O problema acontece novamente por não centralizarmos os dados principais do funcionário em um único lugar.

Existe um jeito, em Java, de relacionarmos uma classe de tal maneira que uma delas **herda** tudo que a outra tem. Isso é uma relação de classe mãe e classe filha. No nosso caso, gostaríamos de fazer com que o `Gerente` tivesse tudo que um `Funcionario` tem, gostaríamos que ela fosse uma **extensão** de `Funcionario`. Fazemos isso por meio da palavra-chave `extends`.

```

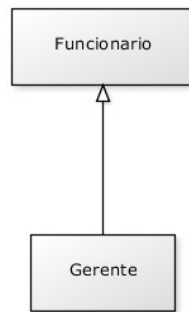
public class Gerente extends Funcionario {
    private int senha;
    private int numeroDeFuncionariosGerenciados;

    public boolean autentica(int senha) {
        if (this.senha == senha) {
            System.out.println("Acesso Permitido!");
            return true;
        } else {
            System.out.println("Acesso Negado!");
            return false;
        }
    }

    // setter da senha omitido
}

```

Em todo momento que criarmos um objeto do tipo `Gerente`, este terá também os atributos definidos na classe `Funcionario`, pois um `Gerente` **é um** `Funcionario`:



```
public class TestaGerente {
    public static void main(String[] args) {
        Gerente gerente = new Gerente();

        // podemos chamar métodos do Funcionario:
        gerente.setNome("João da Silva");

        // e também métodos do Gerente!
        gerente.setSenha(4231);
    }
}
```

Dizemos que a classe `Gerente` **herda** todos os atributos e métodos da classe mãe, no nosso caso, a `Funcionario`. Para ser mais preciso, ela também herda os atributos e métodos privados, porém não consegue acessá-los diretamente. Para acessar um membro privado na filha indiretamente, seria necessário que a mãe expusesse um outro método visível que invocasse esse atributo ou método privado.

SUPER E SUB CLASSE

A nomenclatura mais encontrada é que `Funcionario` é a **superclasse** de `Gerente`, e `Gerente` é a **subclasse** de `Funcionario`. Dizemos também que todo `Gerente` **é um** `Funcionario`. Outra forma é dizer que `Funcionario` é a classe **mãe** de `Gerente`, e `Gerente` é a classe **filha** de `Funcionario`.

E se precisamos acessar os atributos que herdamos? Não gostaríamos de deixar os atributos de `Funcionario`, `public`, pois, dessa maneira, qualquer um poderia alterar os atributos dos objetos desse tipo. Existe um outro modificador de acesso, o `protected`, o qual fica entre o `private` e o `public`. Um atributo `protected` só pode ser acessado (visível) pela própria classe, suas subclasses e classes encontradas no mesmo pacote.

```
public class Funcionario {
    protected String nome;
    protected String cpf;
    protected double salario;
    // métodos devem vir aqui
}
```

SEMPRE USAR PROTECTED?

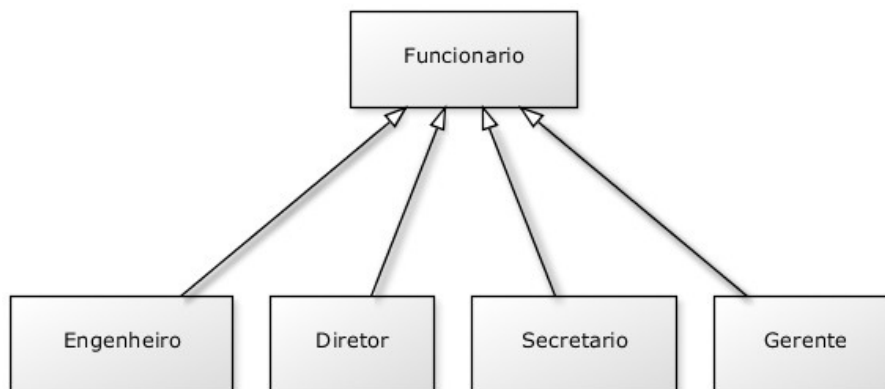
Então por que usar `private` ? Depois de um tempo programando orientado a objetos, você começará a sentir que nem sempre é uma boa ideia deixar a classe filha acessar os atributos da classe mãe, pois isso quebra um pouco a percepção de que só aquela classe deveria manipular seus atributos. Essa é uma discussão um tanto mais avançada.

Além disso, não só as subclasses, mas também as outras classes que se encontram no mesmo pacote podem acessar os atributos `protected` . Veja outras alternativas ao `protected` no exercício de discussão em sala de aula juntamente com o instrutor.

Da mesma maneira, podemos ter uma classe `Diretor` que estenda `Gerente` , e a classe `Presidente` pode estender diretamente de `Funcionario` .

Fique claro que essa é uma decisão de negócio. Se `Diretor` estenderá de `Gerente` ou não, dependerá se, para você, *Diretor é um Gerente* .

Uma classe pode ter várias filhas, mas apenas uma mãe. É a chamada herança simples do Java.



Você pode também fazer o curso data dessa apostila na Caelum



Querendo aprender ainda mais sobre? Esclarecer dúvidas dos exercícios? Ouvir explicações detalhadas com um instrutor?

A Caelum oferece o **curso data** presencial nas cidades de São Paulo, Rio de Janeiro e Brasília, além de turmas incompany.

[Consulte as vantagens do curso Java e Orientação a Objetos](#)

9.2 REESCRITA DE MÉTODO

Todo fim de ano, os funcionários do nosso banco recebem uma bonificação. Os funcionários comuns recebem 10% do valor do salário e os gerentes, 15%.

Vejam como fica a classe `Funcionario` :

```
public class Funcionario {
    protected String nome;
    protected String cpf;
    protected double salario;

    public double getBonificacao() {
        return this.salario * 0.10;
    }
    // métodos
}
```

Se deixarmos a classe `Gerente` como está, ela herdará o método `getBonificacao` .

```
Gerente gerente = new Gerente();
gerente.setSalario(5000.0);
System.out.println(gerente.getBonificacao());
```

O resultado aqui será 500. Não queremos essa resposta, pois o gerente deveria ter 750 de bônus nesse caso. Para consertar isso, uma das opções seria criar um novo método na classe `Gerente` chamado, por exemplo, `getBonificacaoDoGerente` . O problema é que teríamos dois métodos em `Gerente` , confundindo bastante quem for usar essa classe, além disso, cada um dá uma resposta diferente.

No Java, quando herdamos um método, podemos alterar seu comportamento. Podemos **reescrever** (reescrever, sobrescrever, *override*) esse método:

```
public class Gerente extends Funcionario {
    int senha;
    int numeroDeFuncionariosGerenciados;
```

```

    public double getBonificacao() {
        return this.salario * 0.15;
    }
    // ...
}

```

Agora o método está correto para o `Gerente`. Refaça o teste e veja que o valor impresso é o correto (750):

```

Gerente gerente = new Gerente();
gerente.setSalario(5000.0);
System.out.println(gerente.getBonificacao());

```

A ANOTAÇÃO `@Override`

Há como deixar explícito no seu código que determinado método é a reescrita de um método da classe mãe dele. Podemos fazê-lo colocando `@Override` em cima do método. Isso é chamado **anotação**. Existem diversas anotações, e cada uma terá um efeito diferente sobre seu código.

```

@Override
public double getBonificacao() {
    return this.salario * 0.15;
}

```

Repare que, por questões de compatibilidade, isso não é obrigatório. Mas caso um método esteja anotado com `@Override`, ele necessariamente precisa estar reescrevendo um método da classe mãe.

9.3 INVOCANDO O MÉTODO REESCRITO

Depois de reescrito, não podemos mais chamar o método antigo que fora herdado da classe mãe: realmente alteramos o seu comportamento. Mas podemos invocá-lo no caso de estarmos dentro da classe.

Imagine que, para calcular a bonificação de um `Gerente`, devamos fazer igual ao cálculo de um `Funcionario`, porém adicionando R\$ 1000. Poderíamos fazer assim:

```

public class Gerente extends Funcionario {
    int senha;
    int numeroDeFuncionariosGerenciados;

    public double getBonificacao() {
        return this.salario * 0.10 + 1000;
    }
    // ...
}

```

Aqui teríamos um problema: o dia que o `getBonificacao` do `Funcionario` mudar, precisaremos

mudar o método do `Gerente` a fim de acompanhar a nova bonificação. Para evitar isso, o `getBonificacao` do `Gerente` pode chamar o do `Funcionario` utilizando a palavra-chave `super`.

```
public class Gerente extends Funcionario {
    int senha;
    int numeroDeFuncionariosGerenciados;

    public double getBonificacao() {
        return super.getBonificacao() + 1000;
    }
    // ...
}
```

Essa invocação procurará o método com o nome `getBonificacao` de uma super classe de `Gerente`. No caso, ele logo encontrará esse método em `Funcionario`.

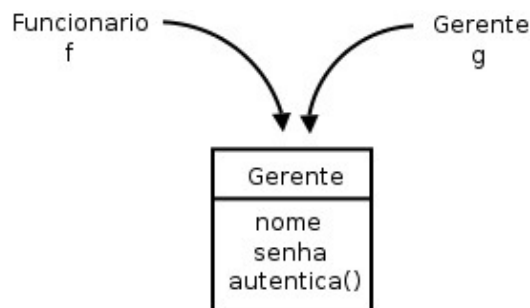
Essa é uma prática comum, pois, em muitos casos, o método reescrito geralmente faz algo a mais que o método da classe mãe. Chamar ou não o método de cima é uma decisão sua e depende do seu problema. Algumas vezes, não faz sentido invocar o método que reescrevemos.

9.4 POLIMORFISMO

O que guarda uma variável do tipo `Funcionario`? Uma referência para um `Funcionario`, nunca o objeto em si.

Na herança, vimos que todo `Gerente` é um `Funcionario`, pois é uma extensão deste. Podemos nos referir a um `Gerente` como sendo um `Funcionario`. Se alguém precisa falar com um `Funcionario` do banco, pode falar com um `Gerente`! Por quê? Pois, `Gerente` **é um** `Funcionario`. Essa é a semântica da herança.

```
Gerente gerente = new Gerente();
Funcionario funcionario = gerente;
funcionario.setSalario(5000.0);
```



Polimorfismo é a capacidade de um objeto poder ser referenciado de várias formas (cuidado, polimorfismo não quer dizer que o objeto fica se transformando, muito pelo contrário, um objeto nasce de um tipo e morre daquele tipo, o que pode mudar é a maneira como nos referimos a ele).

Até aqui tudo bem, mas e se eu tentar:

```
funcionario.getBonificacao();
```

Qual é o retorno desse método? 500 ou 750? No Java, a invocação de método sempre será **decidida em tempo de execução**. O Java procurará o objeto na memória e, aí sim, decidirá qual método deve ser chamado, sempre relacionando com sua classe de verdade, e não com a que estamos usando para referenciá-lo. Apesar de estarmos nos referenciando a esse `Gerente` como sendo um `Funcionario`, o método executado é o do `Gerente`. O retorno é 750.

Parece estranho criar um gerente e referenciá-lo como apenas um funcionário. Por que faríamos isso? Na verdade, a situação que costuma aparecer é a que temos um método que recebe um argumento do tipo `Funcionario`:

```
class ControleDeBonificacoes {
    private double totalDeBonificacoes = 0;

    public void registra(Funcionario funcionario) {
        this.totalDeBonificacoes += funcionario.getBonificacao();
    }

    public double getTotalDeBonificacoes() {
        return this.totalDeBonificacoes;
    }
}
```

E em algum lugar da minha aplicação (ou no `main`, se for apenas para testes):

```
ControleDeBonificacoes controle = new ControleDeBonificacoes();

Gerente funcionario1 = new Gerente();
funcionario1.setSalario(5000.0);
controle.registra(funcionario1);

Funcionario funcionario2 = new Funcionario();
funcionario2.setSalario(1000.0);
controle.registra(funcionario2);

System.out.println(controle.getTotalDeBonificacoes());
```

Repare que conseguimos passar um `Gerente` para um método que recebe um `Funcionario` como argumento. Pense em uma porta na agência bancária com o seguinte aviso: "Permitida a entrada apenas de funcionários". Um gerente pode passar nessa porta? Sim, pois `Gerente` **é um** `Funcionario`.

Qual será o valor resultante? Não importa que dentro do método `registra` o `ControleDeBonificacoes` receba `Funcionario`. Quando ele receber um objeto que realmente é um `Gerente`, o seu método reescrito será invocado. Reafirmando: **não importa como nos referenciamos a um objeto, o método a ser invocado é sempre o do próprio objeto**.

No dia em que criarmos uma classe `Secretaria`, por exemplo, que é filha de `Funcionario`, precisaremos mudar a classe de `ControleDeBonificacoes`? Não. Basta a classe `Secretaria`

reescrever os métodos que lhe parecerem necessários. É exatamente esse o poder do polimorfismo juntamente com a reescrita de método: diminuir o acoplamento entre as classes para evitar que novos códigos resultem em modificações em inúmeros lugares.

Repare que quem criou `ControleDeBonificacoes` pode nunca ter imaginado a criação da classe `Secretaria` ou `Engenheiro`. Contudo, não será necessário reimplementar esse controle em cada nova classe: reaproveitamos aquele código.

HERANÇA VERSUS ACOPLAMENTO

Note que o uso de herança **aumenta** o acoplamento entre as classes, isto é, o quanto uma classe depende de outra. A relação entre as classes mãe e filha é muito forte e isso acaba fazendo com que o programador das classes filhas tenha de conhecer a implementação da classe mãe, e vice-versa – fica difícil fazer uma mudança pontual no sistema.

Por exemplo, imagine se mudássemos algo na nossa classe `Funcionario`, mas não quiséssemos que todos os funcionários sofressem a mesma mudança. Precisaríamos passar por cada uma das filhas de `Funcionario`, verificando se ela se comporta como deveria, ou se deveríamos sobrescrever o tal método modificado.

Esse é um problema da herança, e não do polimorfismo, que resolveremos mais tarde com a ajuda de Interfaces.

Seus livros de tecnologia parecem do século passado?



Conheça a **Casa do Código**, uma **nova** editora, com autores de destaque no mercado, foco em **ebooks** (PDF, epub, mobi), preços **imbatíveis** e assuntos **atuais**.

Com a curadoria da **Caelum** e excelentes autores, é uma abordagem **diferente** para livros de tecnologia no Brasil.

[Casa do Código, Livros de Tecnologia.](http://Casa do Código, Livros de Tecnologia)

9.5 UM OUTRO EXEMPLO

Imagine que modelaremos um sistema para a faculdade que controle as despesas com funcionários e

professores. Nosso funcionário fica assim:

```
public class EmpregadoDaFaculdade {
    private String nome;
    private double salario;
    public double getGastos() {
        return this.salario;
    }
    public String getInfo() {
        return "nome: " + this.nome + " com salário " + this.salario;
    }
    // métodos de get, set e outros
}
```

O gasto que temos com o professor não é apenas o seu salário. Temos de somar um bônus de dez reais por hora/aula. O que fazemos então? Reescrevemos o método. Da mesma forma que o `getGastos` é diferente, o `getInfo` também o será, pois temos de mostrar as horas/aula também.

```
public class ProfessorDaFaculdade extends EmpregadoDaFaculdade {
    private int horasDeAula;
    public double getGastos() {
        return this.getSalario() + this.horasDeAula * 10;
    }
    public String getInfo() {
        String informacaoBasica = super.getInfo();
        String informacao = informacaoBasica + " horas de aula: "
            + this.horasDeAula;

        return informacao;
    }
    // métodos de get, set e outros que forem necessários
}
```

A novidade aqui é a palavra-chave `super`. Apesar do método ter sido reescrito, gostaríamos de acessar o método da classe mãe para não ter de copiar e colocar o conteúdo desse método e depois concatenar com a informação das horas de aula.

Como tiramos proveito do polimorfismo? Imagine que tenhamos uma classe de relatório:

```
public class GeradorDeRelatorio {
    public void adiciona(EmpregadoDaFaculdade f) {
        System.out.println(f.getInfo());
        System.out.println(f.getGastos());
    }
}
```

Poderíamos passar para nossa classe qualquer `EmpregadoDaFaculdade` ! Funcionaria tanto para professor quanto para funcionário comum.

Suponhamos que um certo dia, muito depois de terminar essa classe de relatório, resolvêssemos aumentar nosso sistema e colocar uma classe nova que representa o `Reitor`. Como ele também é um `EmpregadoDaFaculdade`, será que precisaríamos alterar algo na nossa classe de `Relatorio`? Não. Essa é a ideia! Quem programou a classe `GeradorDeRelatorio` nunca imaginou que existiria uma classe `Reitor` e, mesmo assim, o sistema funcionaria.

```

public class Reitor extends EmpregadoDaFaculdade {
    // informações extras
    public String getInfo() {
        return super.getInfo() + " e ele é um reitor";
    }
    // não sobrescrevemos o getGastos!!!
}

```

9.6 UM POUCO MAIS...

- Se não houvesse herança em Java, como você poderia reaproveitar o código de outra classe?
- Uma discussão muito atual é sobre o abuso no uso da herança. Algumas pessoas usam herança apenas para reaproveitar o código, quando poderiam ter feito uma **composição**. Procure sobre herança versus composição.
- Mesmo depois de reescrever um método da classe mãe, a classe filha ainda pode acessar o método antigo. Isso é feito por meio da palavra-chave `super.método()`. Algo parecido ocorre entre os construtores das classes, o quê?

MAIS SOBRE O MAU USO DA HERANÇA

No blog da Caelum, existe um artigo interessante abordando esse tópico:

<http://blog.caelum.com.br/2006/10/14/como-nao-aprender-orientacao-a-objetos-heranca/>

James Gosling, um dos criadores do Java, é um crítico do mau uso da herança. Nesta entrevista, ele discute a possibilidade de se utilizar apenas interfaces e composição, eliminando a necessidade da herança:

<http://www.artima.com/intv/gosling3P.html>

9.7 EXERCÍCIOS: HERANÇA E POLIMORFISMO

1. Teremos mais de um tipo de conta no nosso sistema, então precisaremos de uma nova tela para cadastrar os diferentes tipos de conta. Essa tela já está pronta, e, para utilizá-la, só precisamos alterar a classe que estamos chamando no método `main()` no `TestaContas.java`:

```

package br.com.caelum.contas.main;

import br.com.caelum.javafx.api.main.SistemaBancario;

public class TestaContas {

    public static void main(String[] args) {
        SistemaBancario.mostraTela(false);
    }
}

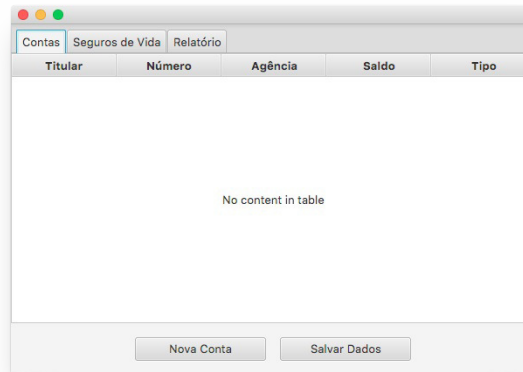
```

```

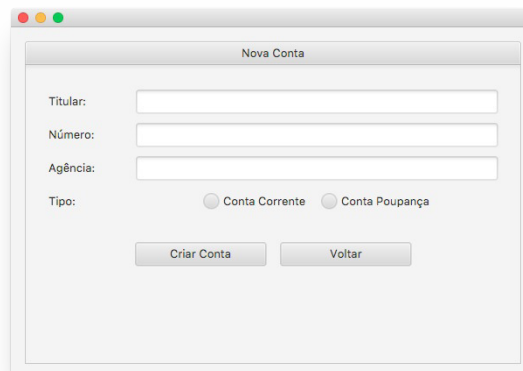
        // TelaDeContas.main(args);
    }
}

```

2. Ao rodar a classe `TestaContas` agora, teremos a tela abaixo:



Entraremos na tela de criação de contas com o objetivo de verificar o que precisaremos implementar para o sistema funcionar. Desse modo, clique no botão **Nova Conta**. A seguinte tela aparecerá:



Podemos perceber que, além das informações que já tínhamos na conta, temos agora o tipo: se queremos uma conta-corrente ou uma conta poupança. Então, criemos as classes correspondentes.

- Crie a classe `ContaCorrente` no pacote `br.com.caelum.contas.modelo` e faça com que ela seja filha da classe `Conta`.
- Crie a classe `ContaPoupanca` no pacote `br.com.caelum.contas.modelo` e faça com que ela seja filha da classe `Conta`.

3. Precisamos pegar os dados da tela para conseguirmos criar a conta correspondente. Na classe

ManipuladorDeContas , alteraremos o método `criaConta` . Atualmente, somente criamos uma nova conta com os dados direto no código. Façamos com que agora os dados sejam recuperados da tela e colocados na nova conta. Para fazer isso, utilize o objeto do tipo `evento` que é exigido como parâmetro do método `criaConta` .

Dicas: a seguir, algumas informações importantes sobre a classe `Evento` (a qual é responsável pela tela Nova Conta):

- Para obter o conteúdo do campo `agencia` , invoque o método `getString("agencia");` ;
- Para obter o conteúdo do campo `numero` , invoque o método `getInt("numero");` ;
- Para obter o conteúdo do campo `titular` , invoque o método `evento.getString("titular");` .
- Observe, na figura anterior, que agora precisamos escolher que tipo de conta queremos criar (conta-corrente ou conta poupança) e, portanto, teremos de recuperar o tipo da conta escolhido e criar a conta correspondente.

Para isso, ao invés de criar um objeto do tipo 'Conta', usaremos o método `getSelecioneadoNoRadio` do objeto `evento` com o intuito de pegar o tipo, fazer um `if` para verificar e só depois criar o objeto do tipo correspondente. A seguir, um trecho do código:

```
public void criaConta(Evento evento){
    String tipo = evento.getSelecioneadoNoRadio("tipo");
    if (tipo.equals("Conta Corrente")) {
        //complete o código
    }
}
```

4. Apesar de já conseguirmos criar os dois tipos de contas, nossa lista não consegue exibir o tipo de cada conta na lista da tela inicial. Para resolver isso, podemos criar um método `getTipo` em cada uma das classes que representam nossas contas, fazendo com que a conta-corrente devolva a string "Conta Corrente" e a conta poupança devolva a string "Conta Poupança".
5. **Atenção!** Altere os métodos `saca` e `deposita` para buscarem o campo `valorOperacao` ao invés de apenas `valor` na classe `ManipuladorDeContas` .
6. Mudemos o comportamento da operação de saque de acordo com o tipo de conta que estiver sendo utilizada. Na classe `ManipuladorDeContas` , alteraremos o método `saca` para tirar 10 centavos de cada saque em uma conta-corrente. A seguir, um trecho do código:

```
public void saca(Evento evento) {
    double valor = evento.getDouble("valorOperacao");
    if (this.conta.getTipo().equals("Conta Corrente")){
        //complete o código
    }
}
```

Dica: ao tentarmos chamar o método `getTipo`, o Eclipse reclamou que este não existe na classe `Conta`, apesar de existir nas classes filhas. O que fazer para resolver isso?

7. O código compila, mas temos um outro problema. A lógica do nosso saque vazou para a classe `ManipuladorDeContas`. Se algum dia precisarmos alterar o valor da taxa no saque, teríamos que mudar em todos os lugares onde fazemos uso do método `saca`. Essa lógica deveria estar encapsulada dentro do método `saca` de cada conta. Como resolver isso?

Dica: repare que, a fim de acessar o atributo `saldo` herdado da classe `Conta`, **você precisará mudar o modificador de visibilidade de saldo para `protected`**.

Agora que a lógica está encapsulada, você precisa corrigir o método `saca` da classe `ManipuladorDeContas`.

Perceba que, neste momento, tratamos a conta de forma genérica!

8. Rode a classe `TestaContas`, adicione uma conta de cada tipo e veja se o tipo é apresentado corretamente na lista de contas da tela inicial.

Agora, clique na conta-corrente apresentada na lista para abrir a tela de detalhes de contas. Teste as operações de saque e depósito e perceba que a conta apresenta o comportamento de uma conta-corrente, conforme o esperado.

E se tentarmos realizar uma transferência da conta-corrente para a conta poupança? O que acontece?

9. Agora você precisa implementar o método `transfere` na classe `Conta` e na classe `ManipuladorDeContas`. Para tal, observe o seguinte:

- O método `transfere` da classe `Conta` deve receber, como parâmetro, duas variáveis, uma referente ao valor a ser transferido, e outra para a conta de destino.
- O método `transfere` da classe `ManipuladorDeContas` deve existir para fazer o vínculo entre a tela e a classe `Conta`, assim ele deve receber um objeto do tipo `Evento` como parâmetro.
- No corpo do método, por meio do objeto do tipo `Evento`, deve-se invocar o método `getSelecioneadoNoCombo("destino")`;
- Em seguida, por meio do objeto do tipo `Conta`, deve-se chamar o método `transfere` da classe `Conta` para que a transferência seja, de fato, realizada.

Rode de novo a aplicação e teste a operação de transferência.

10. Considere o código abaixo:

```
Conta c = new Conta();
```

```
ContaCorrente cc = new ContaCorrente();
ContaPoupanca cp = new ContaPoupanca();
```

Se o mudarmos para:

```
Conta c = new Conta();
Conta cc = new ContaCorrente();
Conta cp = new ContaPoupanca();
```

Compila? Roda? O que muda? Qual é a utilidade disso? Realmente, essa não é a maneira mais útil do polimorfismo. Porém, existe uma utilidade ao declararmos uma variável de um tipo menos específico do que o objeto realmente é, como fazemos na classe `ManipuladorDeContas`.

É **extremamente importante** perceber que não importa como nos referimos a um objeto, o método a ser invocado é sempre o mesmo! A JVM descobrirá, em tempo de execução, qual deve ser invocado, dependendo do tipo daquele objeto, e não considerando como fazemos referência a ele.

11. (Opcional) A nossa classe `Conta` devolve a palavra "Conta" no método `getTipo`. Use a palavra-chave `super` nos métodos `getTipo` reescritos nas classes filhas para não ter de reescrever a palavra "Conta" ao devolver os textos "Conta Corrente" e "Conta Poupança".
12. (Opcional) Se você precisasse criar uma classe `ContaInvestimento`, e seu método `saca` fosse complicadíssimo, precisaria alterar a classe `ManipuladorDeContas`?

Agora é a melhor hora de aprender algo novo

alura

Se você está gostando dessa apostila, certamente vai aproveitar os **cursos online** que lançamos na plataforma **Alura**. Você estuda a qualquer momento com a **qualidade** Caelum. Programação, Mobile, Design, Infra, Front-End e Business, entre outros! Ex-estudante da Caelum tem 10% de desconto, siga o link!

[Conheça a Alura Cursos Online.](#)

9.8 DISCUSSÕES EM AULA: ALTERNATIVAS AO ATRIBUTO PROTECTED

Discuta com o seu instrutor e colegas alternativas ao uso do atributo `protected` na herança. Preciso realmente afrouxar o encapsulamento do atributo por causa da herança? Como fazer para o atributo continuar `private` na mãe, e as filhas conseguirem, de alguma forma, trabalhar com ele?

CLASSES ABSTRATAS

"Dá-se importância aos antepassados quando já não temos nenhum." -- François Chateaubriand

Ao final deste capítulo, você será capaz de utilizar classes abstratas quando necessário.

10.1 REPETINDO MAIS CÓDIGO?

Recordemos como pode estar nossa classe `Funcionario` :

```
public class Funcionario {  
  
    protected String nome;  
    protected String cpf;  
    protected double salario;  
  
    public double getBonificacao() {  
        return this.salario * 1.2;  
    }  
  
    // outros métodos aqui  
  
}
```

Considere o nosso `ControleDeBonificacao` :

```
public class ControleDeBonificacoes {  
  
    private double totalDeBonificacoes = 0;  
  
    public void registra(Funcionario f) {  
        System.out.println("Adicionando bonificação do funcionario: " + f);  
        this.totalDeBonificacoes += f.getBonificacao();  
    }  
  
    public double getTotalDeBonificacoes() {  
        return this.totalDeBonificacoes;  
    }  
  
}
```

Nosso método `registra` recebe qualquer referência do tipo `Funcionario` , isto é, podem ser objetos do tipo `Funcionario` e quaisquer de seus subtipos: `Gerente` , `Diretor` e, conseqüentemente, alguma nova subclasse que venha a ser escrita sem prévio conhecimento do autor da `ControleDeBonificacao` .

Estamos utilizando aqui a classe `Funcionario` para o polimorfismo. Se não fosse ela, teríamos um

grande prejuízo: precisaríamos criar um método `registra` com o objetivo de receber cada um dos tipos de `Funcionario`, um para `Gerente`, um para `Diretor`, etc. Repare que perder esse poder é muito pior do que a pequena vantagem a qual a herança apresenta ao herdar código.

Porém, em alguns sistemas, como é o nosso caso, usamos uma classe com apenas esses intuitos: economizar um pouco código e ganhar polimorfismo para criar métodos mais genéricos que se encaixem em diversos objetos.

Faz sentido ter uma referência do tipo `Funcionario`? Essa pergunta é diferente de saber se faz sentido ter um objeto do tipo `Funcionario`: neste caso, faz, sim, e é muito útil.

Referenciando `Funcionario`, temos o polimorfismo de referência, já que podemos receber qualquer objeto que seja um `Funcionario`. Porém, dar `new` em `Funcionario` pode não fazer sentido, isto é, não queremos receber um objeto do tipo `Funcionario`, mas, sim, que aquela referência seja ou um `Gerente`, ou um `Diretor`, etc. Algo mais **concreto** que um `Funcionario`.

```
ControleDeBonificacoes cdb = new ControleDeBonificacoes();
Funcionario f = new Funcionario();
cdb.adiciona(f); // faz sentido?
```

Vejamos um outro caso em que não faz sentido ter um objeto daquele tipo, apesar da classe existir: imagine a classe `Pessoa` e duas filhas, `PessoaFisica` e `PessoaJuridica`. Quando puxamos um relatório de nossos clientes (uma array de `Pessoa`, por exemplo), queremos que cada um deles seja ou uma `PessoaFisica` ou uma `PessoaJuridica`. A classe `Pessoa`, nesse caso, estaria sendo usada apenas para ganhar o polimorfismo e herdar algumas coisas: não faz sentido permitir instanciá-la.

Para resolver esses problemas, temos as classes abstratas.

Editora Casa do Código com livros de uma forma diferente



Editoras tradicionais pouco ligam para ebooks e novas tecnologias. Não dominam tecnicamente o assunto para revisar os livros a fundo. Não têm anos de experiência em didáticas com cursos.

Conheça a **Casa do Código**, uma editora diferente, com curadoria da **Caelum** e obsessão por livros de qualidade a preços justos.

[Casa do Código, ebook com preço de ebook.](#)

10.2 CLASSE ABSTRATA

O que exatamente vem a ser a nossa classe `Funcionario` ? Nossa empresa tem apenas `Diretores` , `Gerentes` , `Secretárias` , etc. Ela é uma classe que apenas idealiza um tipo, define somente um rascunho.

Para o nosso sistema, é inadmissível um objeto ser apenas do tipo `Funcionario` (pode existir um sistema em que faça sentido ter objetos do tipo `Funcionario` ou apenas `Pessoa` , mas, no nosso caso, não).

Utilizamos a palavra-chave `abstract` para impedir que ela possa ser instanciada. Esse é o efeito direto de se usar o modificador `abstract` na declaração de uma classe:

```
public abstract class Funcionario {  
  
    protected double salario;  
  
    public double getBonificacao() {  
        return this.salario * 1.2;  
    }  
  
    // outros atributos e métodos comuns a todos Funcionarios  
}
```

E no meio de um código:

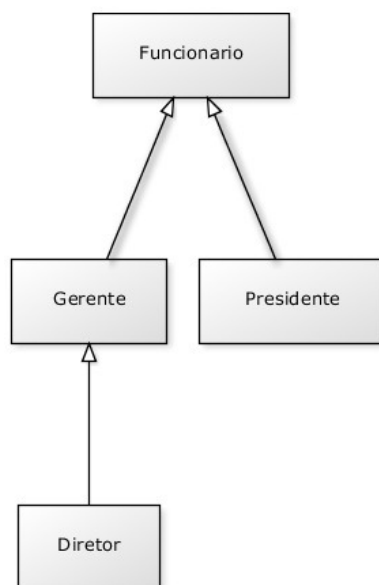
```
Funcionario f = new Funcionario(); // não compila!!!
```

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:  
    Cannot instantiate the type Funcionario  
  
    at br.com.caelum.empresa.TestaFuncionario.main(TestaFuncionario.java:5)
```

O código acima não compila. O problema é instanciar a classe – criar referência você pode. Se ela não pode ser instanciada, para que serve? Serve para o polimorfismo e herança dos atributos e métodos, que são recursos muito poderosos, como já vimos.

Então, herdemos essa classe reescrevendo o método `getBonificacao` :

```
public class Gerente extends Funcionario {  
  
    public double getBonificacao() {  
        return this.salario * 1.4 + 1000;  
    }  
}
```



Mas qual é a real vantagem de uma classe abstrata? Poderíamos ter feito isso com uma herança comum. Por enquanto, a única diferença é que não podemos instanciar um objeto do tipo `Funcionario`, que já é de grande valia, dando mais consistência ao sistema.

Fique claro que a nossa decisão de transformar `Funcionario` em uma classe abstrata dependeu do nosso domínio. Pode ser que, em um sistema com classes similares, faça sentido uma classe análoga a `Funcionario` ser concreta.

10.3 MÉTODOS ABSTRATOS

Se o método `getBonificacao` não fosse reescrito, ele seria herdado da classe mãe, fazendo com que devolvesse o salário mais 20%.

Levando em consideração que cada funcionário em nosso sistema tem uma regra totalmente diferente a fim de ser bonificado, faz algum sentido ter esse método na classe `Funcionario`? Será que existe uma bonificação padrão para todo tipo de `Funcionario`? Parece que não, cada classe filha terá um método diferente de bonificação, pois, de acordo com nosso sistema, não existe uma regra geral: queremos que cada pessoa a qual escreve a classe de um `Funcionario` diferente (subclasses de `Funcionario`) reescreva o método `getBonificacao` de acordo com as suas regras.

Poderíamos, então, jogar fora esse método da classe `Funcionario`? O problema é que, se ele não existisse, não poderíamos chamar o método apenas com uma referência a um `Funcionario`, pois ninguém garante que essa referência aponta para um objeto o qual tem esse método. Será que, dessa maneira, devemos retornar um código como um número negativo? Isso não resolve o problema: se esquecermos de reescrever esse método, teremos dados errados sendo utilizados como bônus.

Em Java, existe um recurso no qual, em uma classe abstrata, podemos escrever que determinado método será **sempre** escrito pelas classes filhas. Isto é, um **método abstrato**.

Ele indica que todas as classes filhas (concretas, ou seja, não abstratas) devem reescrever esse método, ou não compilarão. É como se você herdasse a responsabilidade de ter aquele método.

COMO DECLARAR UM MÉTODO ABSTRATO

Às vezes, não fica claro como declarar um método abstrato.

Basta escrever a palavra-chave `abstract` na sua assinatura e colocar um ponto e vírgula em vez de abrir e fechar chaves!

```
public abstract class Funcionario {  
  
    public abstract double getBonificacao();  
  
    // outros atributos e métodos  
  
}
```

Repare que não colocamos o corpo do método e usamos a palavra-chave `abstract` para defini-lo. Por que não colocar corpo algum? Porque esse método nunca será chamado. Sempre que alguém chamar o método `getBonificacao`, cairá em uma das suas filhas a qual realmente escreveu o método.

Qualquer classe que estender a classe `Funcionario` será obrigada a reescrever esse método, tornando-o concreto. Se não o reescreverem, um erro de compilação ocorrerá.

O método do `ControleDeBonificacao` estava assim:

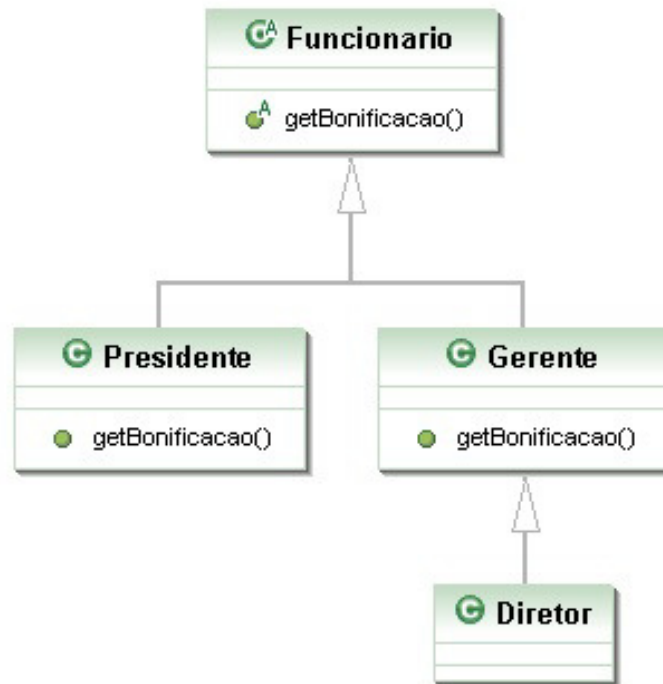
```
public void registra(Funcionario f) {  
    System.out.println("Adicionando bonificação do funcionario: " + f);  
    this.totalDeBonificacoes += f.getBonificacao();  
}
```

Como posso acessar o método `getBonificacao` se ele não existe na classe `Funcionario`?

Já que o método é abstrato, **com certeza** suas subclasses têm esse método, garantindo que essa invocação de método não falhará. Basta pensar: uma referência do tipo `Funcionario` nunca aponta para um objeto que não tem o método `getBonificacao`, pois não é possível instanciar uma classe abstrata, apenas as concretas. Um método abstrato obriga a classe na qual ele se encontra a ser abstrata, assegurando a compilação coerente do código acima.

10.4 AUMENTANDO O EXEMPLO

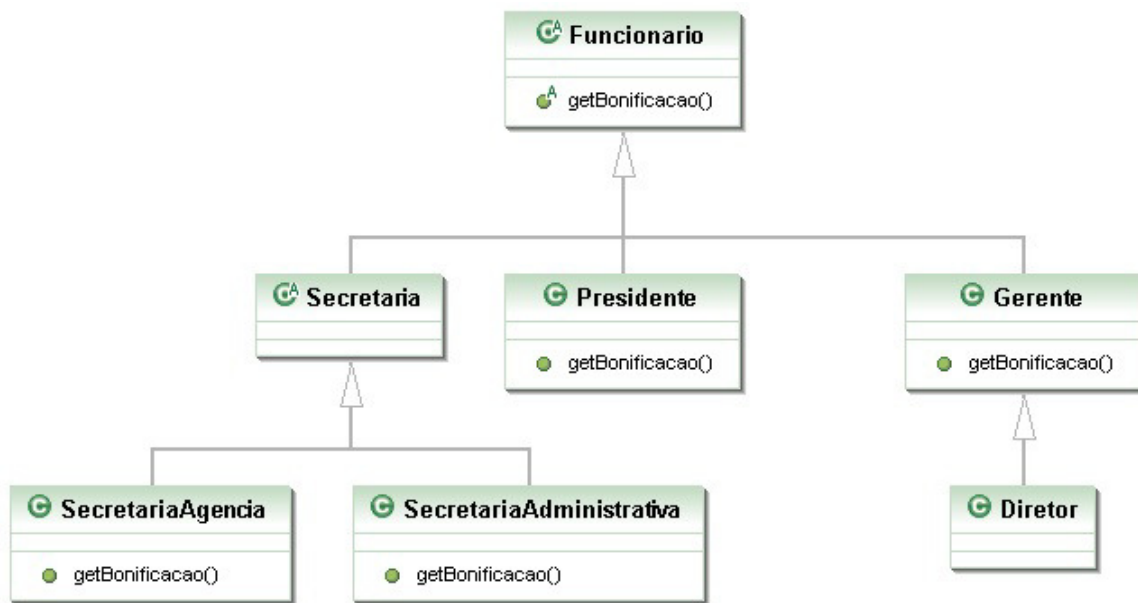
E se, no nosso exemplo de empresa, tivéssemos o próximo diagrama de classes com os seguintes métodos:



Ou seja, tenho a classe abstrata `Funcionario` com o método abstrato `getBonificacao()`; as classes `Gerente` e `Presidente` estendem `Funcionario` e implementam o método `getBonificacao()`; e, por fim, a classe `Diretor`, que estende `Gerente`, mas não implementa o método `getBonificacao()`.

Essas classes compilarão? Rodarão?

A resposta é sim. E, além de tudo, farão exatamente o que nós queremos, pois quando `Gerente` e `Presidente` têm os métodos perfeitamente implementados, a classe `Diretor`, a qual não os tem, usará a implementação herdada de `Gerente`.



E esse diagrama em que incluímos uma classe abstrata `Secretaria` sem o método `getBonificacao`, a qual é estendida por mais duas classes (`SecretariaAdministrativa`, `SecretariaAgencia`) que, por sua vez, implementam o método `getBonificacao`, compilará? Rodará?

De novo, a resposta é sim, pois `Secretaria` é uma classe abstrata. Por isso, o Java tem certeza de que ninguém conseguirá instanciá-la e tampouco chamar o seu método `getBonificacao`. Lembrando que, nesse caso, não precisamos nem ao menos escrever o método abstrato `getBonificacao` na classe `Secretaria`.

Se eu não reescrever um método abstrato da minha classe mãe, o código não compilará. Mas posso, em vez disso, declarar a classe como abstrata!

JAVA.IO

Classes abstratas não têm nenhum segredo de aprendizado, mas quem está aprendendo orientação a objetos pode ter uma enorme dificuldade para saber quando utilizá-las, o que é muito normal.

Estudaremos o pacote `java.io`, que usa bastantes classes abstratas, sendo um exemplo real de uso desse recurso (classe `InputStream` e suas filhas) e, assim, melhorando o seu entendimento.

Já conhece os cursos online Alura?

alura

A **Alura** oferece centenas de **cursos online** em sua plataforma exclusiva de ensino que favorece o aprendizado com a **qualidade** reconhecida da Caelum.

Você pode escolher um curso nas áreas de Programação, Front-end, Mobile, Design & UX, Infra, Business, entre outras, com um plano que dá acesso a todos os cursos. Ex-estudante da Caelum tem 10% de desconto neste link!

[Conheça os cursos online Alura.](#)

10.5 PARA SABER MAIS...

- Uma classe que estende uma classe normal também pode ser abstrata. Ela não poderá ser instanciada, mas sua classe pai, sim!
- Uma classe abstrata não precisa necessariamente ter um método abstrato.

10.6 EXERCÍCIOS: CLASSES ABSTRATAS

1. Repare que a nossa classe `Conta` é uma excelente candidata para uma classe abstrata. Por quê? Que métodos seriam interessantes candidatos a serem abstratos?

Transforme a classe `Conta` em abstrata:

```
public abstract class Conta {  
    // ...  
}
```

2. Como a classe `Conta` agora é abstrata, não conseguimos dar `new` nela mais. Se não podemos dar `new` em `Conta`, qual é a utilidade de ter um método que recebe uma referência à `Conta` como argumento? Aliás, posso ter isso?
3. Para entender melhor o `abstract`, comente o método `getTipo()` da `ContaPoupanca`. Dessa forma, ele herdará o método diretamente de `Conta`.

Transforme o método `getTipo()` da classe `Conta` em abstrato. Repare que, ao colocar a palavra-chave `abstract` ao lado do método, o Eclipse rapidamente recomendará a remoção do corpo (body) do método com um quickfix.

Sua classe `Conta` deve ficar parecida com:

```

public abstract class Conta {
    // atributos e métodos que já existiam

    public abstract String getTipo();
}

```

Qual é o problema com a classe `ContaPoupanca` ?

4. Descomente o método `getTipo` na classe `ContaPoupanca` e, se necessário, altere-o para que a classe possa compilar normalmente.
5. (Opcional) Existe outra maneira de a classe `ContaPoupanca` compilar se você não reescrever o método abstrato?
6. (Opcional) Para que ter o método `getTipo` na classe `Conta` se ele não faz nada? O que acontece se simplesmente apagarmos esse método da classe `Conta` e deixarmos o método `getTipo` nas filhas?
7. (Opcional) Posso chamar um método abstrato de dentro de um outro método da própria classe abstrata? Por exemplo, imagine que exista o seguinte método na classe `Conta` :

```

public String recuperaDadosParaImpressao() {
    String dados = "Titular: " + this.titular;
    dados += "\nNúmero: " + this.numero;
    dados += "\nAgência: " + this.agencia;
    dados += "\nSaldo: R$" + this.saldo;
    return dados;
}

```

Poderíamos invocar o `getTipo` dentro desse método? Algo como:

```

dados += "\nTipo: " + this.getTipo();

```


INTERFACES

"Uma imagem vale mil palavras. Uma interface vale mil imagens." -- Ben Shneiderman

Ao final deste capítulo, você será capaz de:

- Dizer o que é uma interface e as diferenças entre herança e implementação;
- Escrever uma interface em Java;
- Utilizá-las como um poderoso recurso para diminuir acoplamento entre as classes.

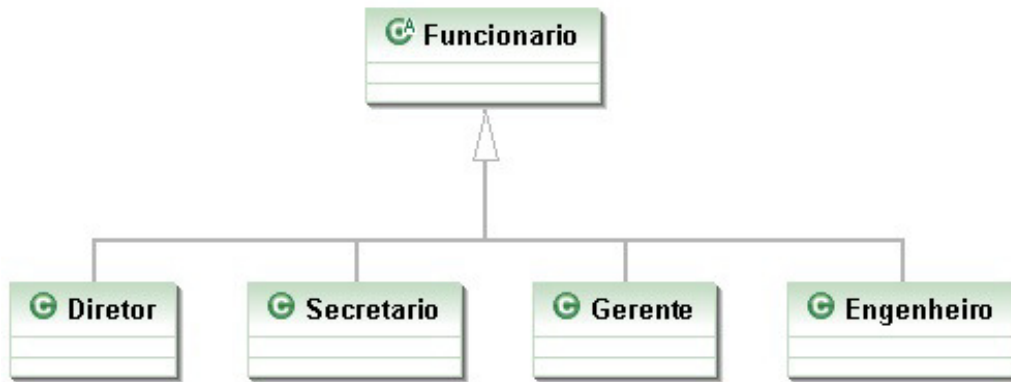
11.1 AUMENTANDO NOSSO EXEMPLO

Imagine que um sistema de controle do banco possa ser acessado pelos diretores do banco, além dos gerentes. Então, teríamos uma classe `Diretor` :

```
public class Diretor extends Funcionario {  
  
    public boolean autentica(int senha) {  
        // Verificar aqui se a senha confere com a recebida como parâmetro.  
    }  
  
}
```

E a classe `Gerente` :

```
public class Gerente extends Funcionario {  
  
    public boolean autentica(int senha) {  
        // Verificar aqui se a senha confere com a recebida como parâmetro.  
        // No caso do gerente, conferir também se o departamento dele  
        // tem acesso.  
    }  
  
}
```



Repare que o método de autenticação de cada tipo de `Funcionario` pode variar muito. Mas vamos aos problemas. Considere o `SistemaInterno` e seu controle: precisamos receber um `Diretor` ou `Gerente` como argumento, verificar se ele se autentica e colocá-lo dentro do sistema.

```

public class SistemaInterno {

    public void login(Funcionario funcionario) {
        // Invocar o método autentica?
        // Não dá! Nem todo Funcionario o tem.
    }
}
  
```

O `SistemaInterno` aceita qualquer tipo de `Funcionario`, tendo ele acesso ao sistema ou não, mas note que nem todo `Funcionario` tem o método `autentica`. Isso nos impede de chamar esse método com uma referência apenas a `Funcionario` (haveria um erro de compilação). O que fazer, então?

```

public class SistemaInterno {

    public void login(Funcionario funcionario) {
        funcionario.autentica(...); // não compila
    }
}
  
```

Uma possibilidade é criar dois métodos `login` no `SistemaInterno`: um para receber `Diretor`, e outro, `Gerente`. Já vimos que essa não é uma boa escolha. Por quê?

```

public class SistemaInterno {

    // design problemático
    public void login(Diretor funcionario) {
        funcionario.autentica(...);
    }

    // design problemático
    public void login(Gerente funcionario) {
        funcionario.autentica(...);
    }
}
  
```

Cada vez que criarmos uma nova classe de `Funcionario` que é *autenticável*, precisaríamos adicionar um novo método de login no `SistemaInterno`.

MÉTODOS COM MESMO NOME

Em Java, métodos podem ter o mesmo nome desde que não sejam ambíguos, isto é, que exista uma maneira de distingui-los no momento da chamada.

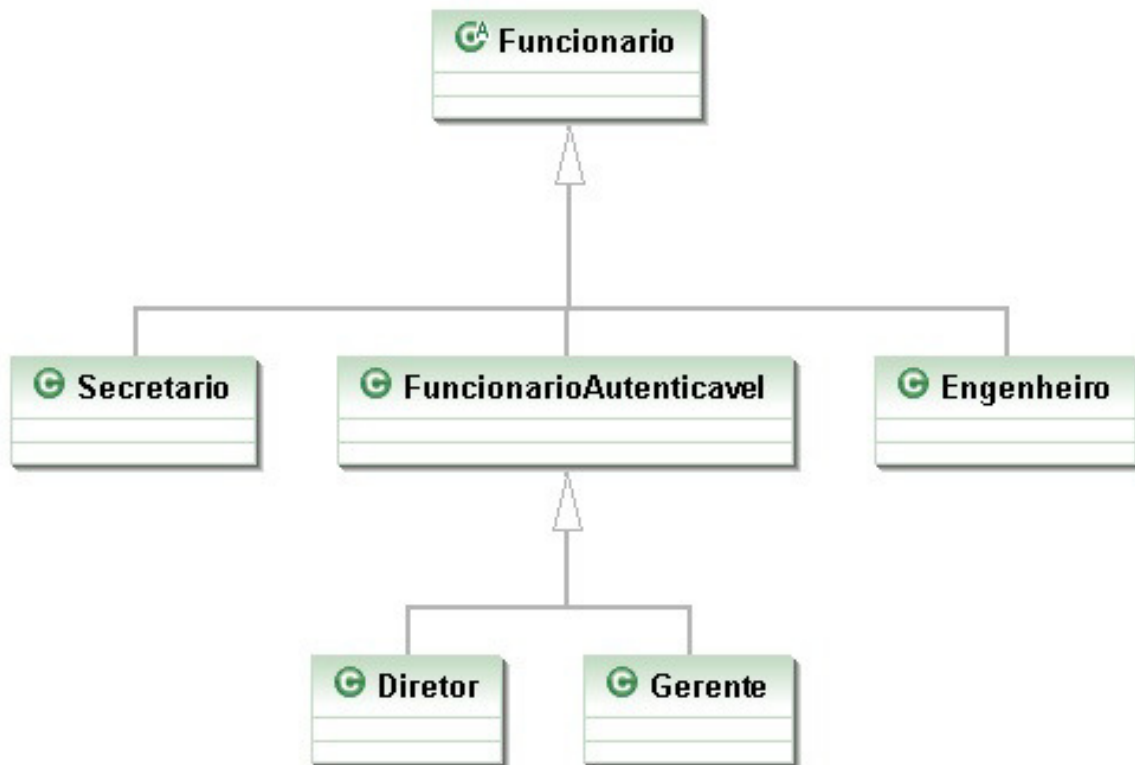
Isso se chama **sobrecarga** de método. (**Overloading**. Não confundir com **overriding**, que é um conceito muito mais poderoso).

Uma solução mais interessante seria criar uma classe no meio da árvore de herança, `FuncionarioAutenticavel`:

```
public class FuncionarioAutenticavel extends Funcionario {  
  
    public boolean autentica(int senha) {  
        // Faz autenticação padrão.  
    }  
  
    // Outros atributos e métodos.  
}
```

As classes `Diretor` e `Gerente` passariam a estender de `FuncionarioAutenticavel`, e o `SistemaInterno` receberia referências desse tipo, como se mostra a seguir:

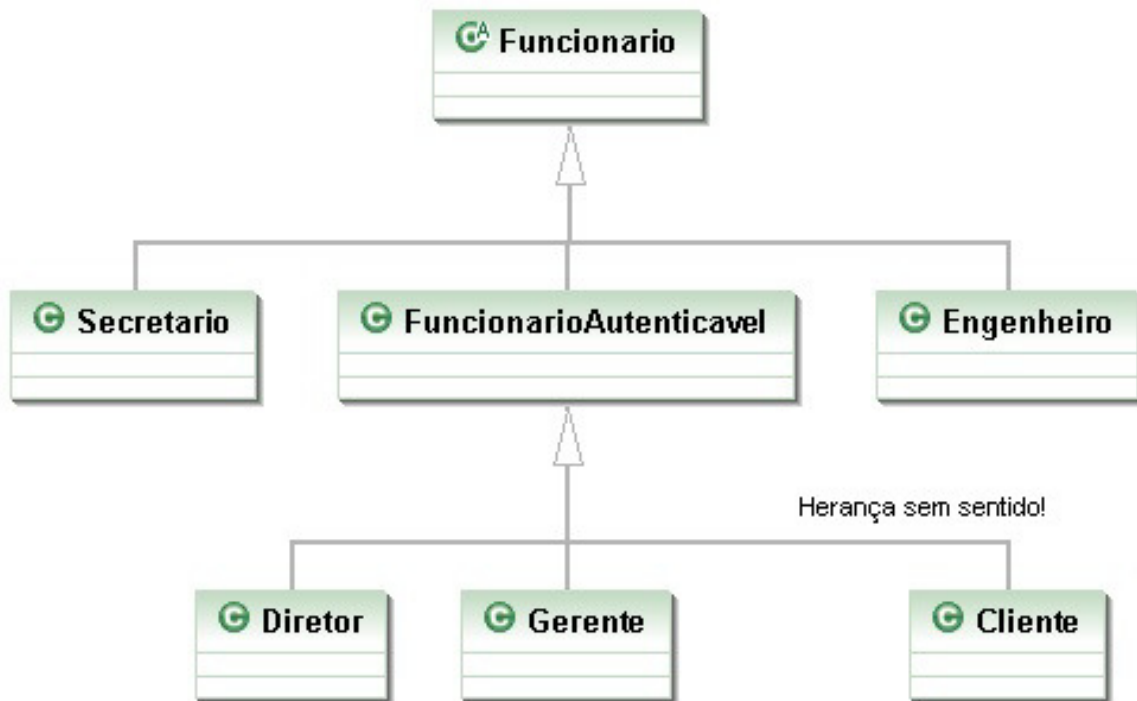
```
public class SistemaInterno {  
  
    public void login(FuncionarioAutenticavel fa) {  
  
        int senha = //Pega senha de um lugar ou de um scanner de polegar.  
  
        // Aqui eu posso chamar o autentica!  
        // Pois, todo FuncionarioAutenticavel o tem.  
        boolean ok = fa.autentica(senha);  
  
    }  
}
```



Repare que `FuncionarioAutenticavel` é um forte candidato à classe abstrata. Além disso, o método `autentica` ainda poderia ser um método abstrato.

O uso de herança resolve esse caso, mas vamos a uma outra situação um pouco mais complexa: precisamos que todos os clientes também tenham acesso ao `SistemaInterno`. O que fazer? Uma opção é criar outro método `login` em `SistemaInterno`, mas já descartamos essa possibilidade anteriormente.

Uma outra opção que é comum entre os novatos é fazer uma herança sem sentido para resolver o problema, por exemplo, fazer `Cliente` extends `FuncionarioAutenticavel`. Realmente resolve o problema, mas trará diversos outros. `Cliente` definitivamente **não é** `FuncionarioAutenticavel`. Se você fizer isso, o `Cliente` terá, por exemplo, um método `getBonificacao`, um atributo `salário` e outros membros que não fazem o menor sentido para essa classe. Não faça herança caso a relação não seja estritamente "é um".



Como resolver essa situação? Note que conhecer a sintaxe da linguagem não é o suficiente, precisamos estruturar/desenhar bem a nossa estrutura de classes.

Você pode também fazer o curso data dessa apostila na Caelum



Querendo aprender ainda mais sobre? Esclarecer dúvidas dos exercícios? Ouvir explicações detalhadas com um instrutor?

A Caelum oferece o **curso data** presencial nas cidades de São Paulo, Rio de Janeiro e Brasília, além de turmas incompany.

[Consulte as vantagens do curso Java e Orientação a Objetos](#)

11.2 INTERFACES

O que precisamos para resolver nosso problema? Arranjar uma forma de poder referenciar `Diretor`, `Gerente` e `Cliente` de uma mesma maneira, isto é, achar um fator comum.

Se houvesse uma forma na qual essas classes garantissem a existência de um determinado método por meio de um contrato, resolveríamos o problema.

Toda classe define dois itens:

- O que uma classe faz (as assinaturas dos métodos);
- Como uma classe faz essas tarefas (o corpo dos métodos e atributos privados).

Podemos criar um "contrato" o qual define tudo o que uma classe deve fazer se quiser ter um determinado status. Imagine:

contrato "Autenticavel":

Quem quiser ser "Autenticavel" precisa saber:
1. Autenticar uma senha, devolvendo um booleano.

Quem quiser pode assinar esse contrato, sendo, assim, obrigado a explicar como será feita essa autenticação. A vantagem é que, se um `Gerente` assinar esse contrato, podemos nos referenciar a um `Gerente` como um `Autenticavel`.

Podemos criar esse contrato em Java!

```
public interface Autenticavel {  
  
    boolean autentica(int senha);  
  
}
```

Chama-se `interface`, pois é a maneira pela qual poderemos conversar com um `Autenticavel`. Interface é a maneira por meio da qual conversamos com um objeto.

Lemos a interface da seguinte maneira: *"quem desejar ser `Autenticavel` precisa saber autenticar recebendo um inteiro e retornando um booleano"*. Ela é um contrato que quem assina se responsabiliza por implementar esses métodos (cumprir o contrato).

Pela ideia base de uma interface, ela pode definir uma série de métodos, mas nunca conter suas implementações. Ela só expõe **o que o objeto deve fazer**, e não **como ele o faz**, nem **o que ele tem**. **Como ele o faz** será definido em uma **implementação** dessa interface.

E o `Gerente` pode "assinar" o contrato, ou seja, **implementar** a interface. No momento em que ele implementa essa interface, precisa escrever os métodos pedidos por ela (muito parecido com o efeito de herdar métodos abstratos, aliás, métodos de uma interface são públicos e abstratos sempre). Para implementar, usamos a palavra-chave `implements` na classe:

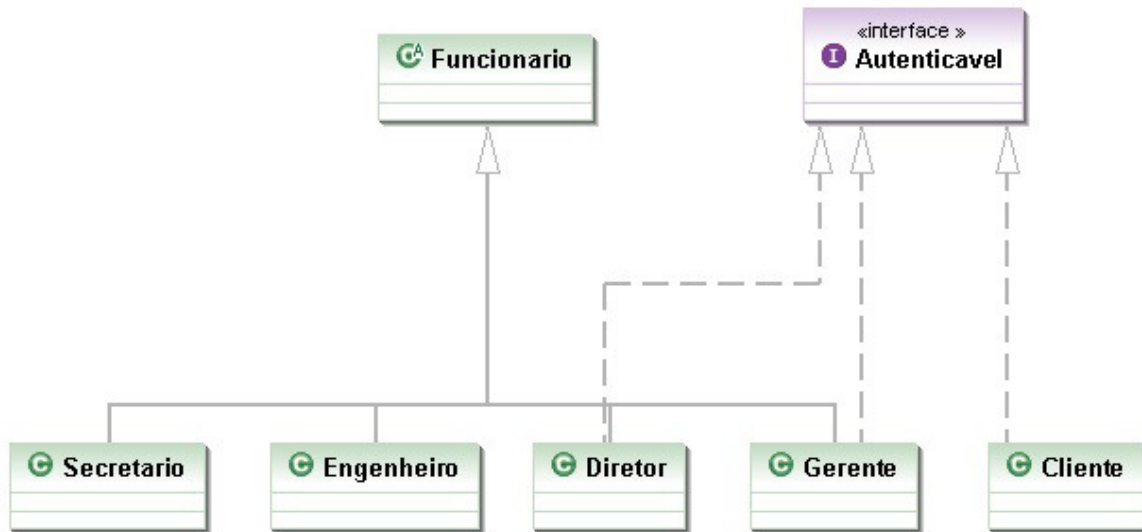
```
public class Gerente extends Funcionario implements Autenticavel {  
  
    private int senha;  
  
    // Outros atributos e métodos.  
  
    public boolean autentica(int senha) {  
        if(this.senha != senha) {  
            return false;  
        }  
    }  
}
```

```

    // Pode fazer outras possíveis verificações como saber se esse
    // departamento do gerente tem acesso ao Sistema.

    return true;
}
}

```



O `implements` pode ser lido da seguinte maneira: "a classe `Gerente` se compromete a ser tratada como `Autenticavel`, sendo obrigada a ter os métodos necessários, definidos neste contrato".

A partir de agora, podemos tratar um `Gerente` como sendo um `Autenticavel`. Ganhamos mais polimorfismo! Temos mais uma forma de referenciar a um `Gerente`. Quando crio uma variável do tipo `Autenticavel`, estou criando uma referência a **qualquer** objeto de uma classe que implemente `Autenticavel`, direta ou indiretamente:

```

Autenticavel a = new Gerente();
// Posso aqui chamar o método autentica!

```

Novamente, a utilização mais comum seria receber por argumento, como no nosso `SistemaInterno`:

```

public class SistemaInterno {

    public void login(Autenticavel a) {
        int senha = // Pega senha de um lugar ou de um scanner de polegar.
        boolean ok = a.autentica(senha);

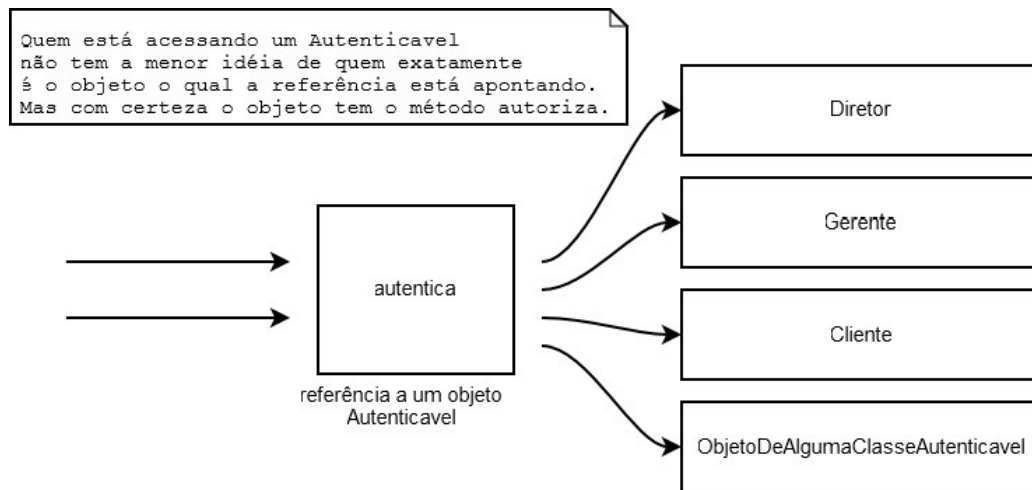
        // Aqui eu posso chamar o autentica!
        // Não necessariamente é um Funcionario!
        // Além do mais, eu não sei que objeto a
        // referência "a" está apontando exatamente! Flexibilidade.
    }

}

```

Pronto! E já podemos passar qualquer `Autenticavel` para o `SistemaInterno`. Então, precisamos fazer com que o `Diretor` também implemente essa interface.

```
public class Diretor extends Funcionario implements Autenticavel {  
    // Métodos e atributos devem obrigatoriamente ter o autentica.  
}
```



Podemos passar um `Diretor`. No dia em que tivermos mais um funcionário com acesso ao sistema, bastará que ele implemente essa interface para se encaixar no sistema.

Qualquer `Autenticavel` passado ao `SistemaInterno` está bom para nós. Repare que pouco importa quem o objeto referenciado realmente é, pois ele tem um método `autentica` o qual é necessário para nosso `SistemaInterno` funcionar corretamente. Aliás, qualquer outra classe que futuramente implemente essa interface poderá ser passada como argumento aqui.

```
Autenticavel diretor = new Diretor();  
Autenticavel gerente = new Gerente();
```

Ou, se achamos que o `Fornecedor` precisa ter acesso, ele só precisará implementar `Autenticavel`. Olhe só o tamanho do desacoplamento: quem escreveu o `SistemaInterno` necessita somente saber que ele é `Autenticavel`.

```
public class SistemaInterno {  
    public void login(Autenticavel a) {  
        // Não importa se ele é um gerente ou diretor,  
        // será que é um fornecedor?  
        // Eu, o programador do SistemaInterno, não me preocupo.  
        // Invocarei o método autentica.  
    }  
}
```

Não faz diferença se é um `Diretor`, `Gerente`, `Cliente` ou qualquer classe que venha por aí.

Basta seguir o contrato! Além do mais, cada `Autenticavel` pode se autenticar de uma maneira completamente diferente de outro.

Lembre-se: a interface define que todos saberão se autenticar (o que ele faz), enquanto a implementação define como exatamente será feito (de que forma ele faz).

A maneira pela qual os objetos se comunicam em um sistema orientado a objetos é muito mais importante do que como eles executam. **O que um objeto faz** é mais importante do que **como ele o faz**. Aqueles que seguem essa regra terão sistemas mais fáceis de manter e modificar. Conforme você já percebeu, essa é uma das ideias principais que queremos passar e, provavelmente, a mais importante de todo esse curso.

MAIS SOBRE INTERFACES: HERANÇA E MÉTODOS DEFAULT

Diferentemente das classes, uma interface pode herdar de mais de uma interface. É como um contrato o qual depende que outros contratos sejam fechados antes daquele valer. Você não herda métodos e atributos, mas, sim, responsabilidades.

Um outro recurso em interfaces são os métodos default a partir do Java 8. Você pode, sim, declarar um método concreto utilizando a palavra `default` ao lado, e suas implementações não precisam necessariamente reescrevê-lo. Veremos que isso acontece, por exemplo, com o método `List.sort` durante o capítulo de coleções. É um truque muito utilizado para poder evoluir uma interface sem quebrar compatibilidade com as implementações anteriores.

11.3 DIFICULDADE NO APRENDIZADO DE INTERFACES

Interfaces representam uma barreira no aprendizado do Java: parece que estamos escrevendo um código o qual não serve para nada, uma vez que teremos essa linha (a assinatura do método) escrita nas nossas classes implementadoras. Essa é uma maneira errada de se pensar. O objetivo do uso de uma interface é deixar seu código mais flexível e possibilitar a mudança de implementação sem maiores traumas. **Não é apenas um código de prototipação ou um cabeçalho!**

Os mais radicais dizem que toda classe deve ser interfaceada, isto é, só devemos nos referir a objetos por intermédio das suas interfaces. Se determinada classe não tem uma interface, ela deveria tê-la. Os autores deste material acham tal medida radical demais, porém o uso de interfaces em vez de herança é amplamente aconselhado. Você pode encontrar mais informações sobre o assunto nos livros *Design Patterns*, *Refactoring* e *Effective Java*.

No livro *Design Patterns*, logo no início, os autores citam duas regras de ouro. Uma é: "evite herança, prefira composição", e a outra: " programe voltado à interface, e não à implementação".

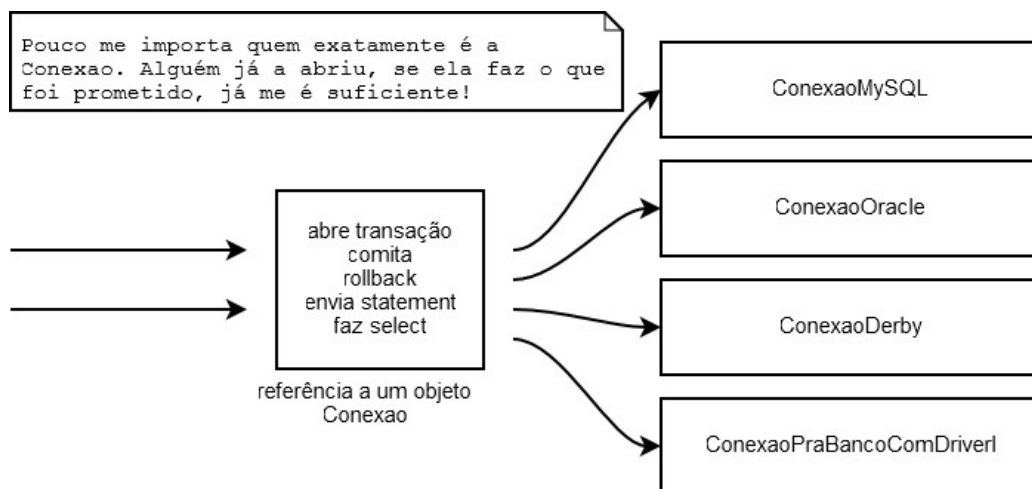
Veremos o uso de interfaces no capítulo de coleções, o que melhorará o entendimento do assunto. O exemplo da interface `Comparable` também é muito esclarecedor, no qual enxergamos o reaproveitamento de código mediante as interfaces, além do encapsulamento. Para o método `Collections.sort()`, pouco importa quem será passado como argumento, pois basta que a coleção seja de objetos comparáveis. Ele pode ordenar `Elefante`, `Conexao` ou `ContaCorrente`, desde que implementem `Comparable`.

11.4 EXEMPLO INTERESSANTE: CONEXÕES COM O BANCO DE DADOS

Como fazer com que todas as chamadas para bancos de dados diferentes respeitem a mesma regra? Usando interfaces!

Imagine uma interface `Conexao` contendo todos os métodos necessários à comunicação e troca de dados com um banco de dados. Cada banco de dados fica encarregado de criar a sua implementação para essa interface.

Quem for usar uma `Conexao` não precisa se importar com qual objeto exatamente está trabalhando, posto que ele cumprirá o papel que toda `Conexao` deve ter. Não importa se é uma conexão com um Oracle ou MySQL.



Apesar do `java.sql.Connection` não trabalhar bem assim, a ideia é muito similar. Porém, as conexões vêm de uma *factory* chamada `DriverManager`.

Conexão a banco de dados está fora do escopo desse treinamento, mas é um dos primeiros tópicos abordados no curso FJ-21 juntamente com DAO.

UM POUCO MAIS...

- Posso substituir toda minha herança por interfaces? Qual é a vantagem e a desvantagem?

Seus livros de tecnologia parecem do século passado?



Conheça a **Casa do Código**, uma **nova** editora, com autores de destaque no mercado, foco em **ebooks** (PDF, epub, mobi), preços **imbatíveis** e assuntos **atuais**.

Com a curadoria da **Caelum** e excelentes autores, é uma abordagem **diferente** para livros de tecnologia no Brasil.

[Casa do Código, Livros de Tecnologia.](#)

11.5 EXERCÍCIOS: INTERFACES

1. Nosso banco precisa tributar dinheiro de alguns bens que nossos clientes possuem. Para isso, criaremos uma interface no pacote `br.com.caelum.contas.modelo` do nosso projeto `fj11-contas` já existente:

- O nome da interface deverá ser `Tributavel` ;
- Deverá ter um único método chamado `getValorImposto()` , que não recebe nada e devolve um `double`.

Lemos essa interface da seguinte maneira: "todos que quiserem ser *tributável* precisam saber retornar *o valor do imposto*, devolvendo um `double`".

Alguns bens são tributáveis e outros não. `ContaPoupanca` não é tributável, já para `ContaCorrente` , você precisa pagar 1% da conta, e o `SeguroDeVida` tem uma taxa fixa de 42 reais mais 2% do valor do seguro.

Assim, para atender a essa nova necessidade, você deve:

- *Alterar* a classe `ContaCorrente` ;
- *Criar* a classe `SeguroDeVida` .

A classe `SeguroDeVida` deverá estar no pacote `br.com.caelum.contas.modelo` e ter os seguintes atributos encapsulados: `valor` (do tipo `double`), `titular` (do tipo `String`) e `numeroApolice` (do tipo `int`).

Dica: na classe `SeguroDeVida`, lembre-se de escrever o método `getTipo` para que o tipo do produto apareça na interface gráfica.

2. Criaremos a classe `ManipuladorDeSeguroDeVida` dentro do pacote `br.com.caelum.contas` para vincular a classe `SeguroDeVida` à tela de criação de seguros. Aquela classe deve ter um atributo do tipo `SeguroDeVida`.

Deve ter também o método `criaSeguro`, que não retorna nada e deve receber um parâmetro do tipo `Evento` para conseguir obter os dados da tela. Use os seguintes métodos da classe `Evento` a fim de pegar estes dados:

- `evento.getInt("numeroApolice");`
- `evento.getString("titular");`
- `evento.getDouble("valor");`

Dica: use os métodos *setters* da classe `SeguroDeVida` para guardar as informações obtidas. Exemplo:

```
this.seguroDeVida.setNumeroApolice(evento.getInt("numeroApolice"));
```

3. Execute a classe `TestaContas` e tente cadastrar um novo seguro de vida. O seguro cadastrado deve aparecer na tabela de seguros de vida.
4. Queremos saber qual o valor total dos impostos de todos os tributáveis. Então criemos a classe `ManipuladorDeTributaveis` dentro do pacote `br.com.caelum.contas`. Crie também o método `calculaImpostos`, que não retorna nada e recebe um parâmetro do tipo `Evento`. Mais pra frente preencheremos o corpo desse método.

Essa classe também deverá ter o atributo encapsulado `total` do tipo `double`.

5. Agora que criamos o tributável, habilitaremos a última aba de nosso sistema. Altere a classe `TestaContas` para passar o valor `true` na chamada do método `mostraTela`.

Observe: agora que temos o seguro de vida funcionando, a tela de relatório já consegue imprimir o valor dos impostos individuais de cada tipo de *Tributavel*.

6. No método `calculaImpostos` da classe `ManipuladorDeTributaveis`, precisamos buscar os valores de impostos de cada `Tributavel`, somá-los e atribuí-los ao atributo `total`. Para isso, usaremos os seguintes métodos da classe `Evento`:

- `getTamanhoDaLista` que deve receber o nome da lista desejada, nesse caso, `"listaTributaveis"`. Esse método retorna a quantidade de tributáveis:

```
evento.getTamanhoDaLista("listaTributaveis");
```

- `getTributavel` retorna um `Tributavel` de uma determinada posição de uma lista, em que precisamos passar o nome da lista e o índice do elemento:

```
evento.getTributavel("listaTributaveis", i);
```

Dica: utilize o comando `for` para percorrer a lista inteira, passando por cada posição.

Por fim, o método `calculaImpostos` deverá invocar o método `getValorImposto()` e acumular o valor do imposto de todos os tributáveis no atributo `total` :

```
total += t.getValorImposto();
```

Repare que, de dentro do `ManipuladorDeTributaveis` , você não pode acessar o método `getSaldo` , por exemplo, pois não há a garantia de que o `Tributavel` a ser passado como argumento tenha esse método. A única certeza é a de que esse objeto tem os métodos declarados na interface `Tributavel` .

É interessante enxergar que as interfaces (como nesse caso, `Tributavel`) costumam ligar classes muito distintas, unindo-as por uma característica que elas têm em comum. No nosso exemplo, `SeguroDeVida` e `ContaCorrente` são entidades completamente distintas, porém ambas possuem a característica de serem tributáveis.

Se amanhã o governo começar a tributar até mesmo `PlanoDeCapitalizacao` , basta que essa classe implemente a interface `Tributavel` ! Repare no grau de desacoplamento que temos: a classe `GerenciadorDeImpostoDeRenda` nem imagina que trabalhará como `PlanoDeCapitalizacao` . Para ela, o único fato importante é que o objeto respeite o contrato de um tributável, isto é, a interface `Tributavel` . Novamente: programe voltado à interface, não à implementação.

Quais os benefícios de manter o código com baixo acoplamento?

7. (Opcional) Crie a classe `TestaTributavel` com um método `main` para testar o nosso exemplo:

```
public class TestaTributavel {  
  
    public static void main(String[] args) {  
        ContaCorrente cc = new ContaCorrente();  
        cc.deposita(100);  
        System.out.println(cc.getValorImposto());  
  
        // testando polimorfismo:  
        Tributavel t = cc;  
        System.out.println(t.getValorImposto());  
    }  
}
```

```
}
```

Tente chamar o método `getSaldo` por meio da referência `t`. O que ocorre? Por quê?

A linha em que atribuímos `cc` a um `Tributavel` é apenas para você enxergar que é possível fazê-lo. Nesse nosso caso, isso não tem uma utilidade. Essa possibilidade foi útil no exercício anterior.

11.6 EXERCÍCIOS OPCIONAIS

Atenção: caso você resolva esse exercício, faça-o em um projeto à parte `conta-interface`, uma vez que usaremos a `Conta` como classe em exercícios futuros.

1. (Opcional) Transforme a classe `Conta` em uma interface.

```
public interface Conta {  
    public double getSaldo();  
    public void deposita(double valor);  
    public void saca(double valor);  
    public void atualiza(double taxaSelic);  
}
```

Adapte `ContaCorrente` e `ContaPoupanca` a essa modificação:

```
public class ContaCorrente implements Conta {  
    // ...  
}  
  
public class ContaPoupanca implements Conta {  
    // ...  
}
```

Algum código terá de ser copiado e colado? Isso é tão ruim?

2. (Opcional) Às vezes, é interessante criarmos uma interface que herde de outras interfaces: aquela é chamada de subinterface, e nada mais é do que um agrupamento de obrigações para a classe que a implementar.

```
public interface ContaTributavel extends Conta, Tributavel {  
}
```

Dessa maneira, quem for implementar essa nova interface precisa implementar todos os métodos herdados das suas superinterfaces (e talvez ainda novos métodos declarados dentro dela):

```
public class ContaCorrente implements ContaTributavel {  
    // métodos  
}  
  
Conta c = new ContaCorrente();  
Tributavel t = new ContaCorrente();
```

Repare que o código pode parecer estranho, pois a interface não declara método algum, só herda os métodos abstratos declarados nas outras interfaces.

Ao mesmo tempo que uma interface pode herdar de mais de uma outra interface, classes só podem ter uma classe mãe (herança simples).

11.7 DISCUSSÃO: FAVOREÇA COMPOSIÇÃO EM RELAÇÃO À HERANÇA

Discuta com o seu instrutor e colegas alternativas à herança. Falaremos de herança versus composição, além de o porquê da herança ser, muitas vezes, considerada maléfica.

Em uma entrevista, James Gosling, pai do Java, fala sobre uma linguagem puramente de delegação e chega a dizer:

Rather than subclassing, just use pure interfaces. It's not so much that class inheritance is particularly bad. It just has problems.

(Tradução livre: "Em vez de fazer subclasses, use simplesmente interfaces. Não é que a herança de classes seja particularmente ruim. Ela só tem problemas.")

<http://www.artima.com/intv/gosling3P.html>

No blog da Caelum, há também um post sobre o assunto:
<http://blog.caelum.com.br/2006/10/14/como-nao-aprender-orientacao-a-objetos-heranca/>

Agora é a melhor hora de aprender algo novo

alura

Se você está gostando dessa apostila, certamente vai aproveitar os **cursos online** que lançamos na plataforma **Alura**. Você estuda a qualquer momento com a **qualidade** Caelum. Programação, Mobile, Design, Infra, Front-End e Business, entre outros! Ex-estudante da Caelum tem 10% de desconto, siga o link!

[Conheça a Alura Cursos Online.](#)